Lab 06: Secret Management Using GPG - Exercise report

Exercise Goal

Learn what secrets are, why they must not be stored in source code, how to protect them with GPG, and the basics of secrets management without specialized tools.

1. Kali Linux update and Install Gnupg

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:4 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [120 kB]
Get:5 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [276 kB]
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [189 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [912 kB]
Fetched 75.9 MB in 1min 27s (870 kB/s)
302 packages can be upgraded. Run 'apt list --upgradable' to see them.

┌──(kali㉿kali)-[~]
└─$ sudo apt install gnupg
gnupg is already the newest version (2.4.9-4).
The following packages were automatically installed and are no longer required:
  aardvark-dns                  libblake3-0      libslirp0                     node-readdirp
  buildah                       libcriu2         libsodium23                   node-set-immediate-shim
  catatonit                     libgo-14-dev     libsubid5                     openjdk-21-jre
  conmon                        libgo23          libteamdctl0                  passt
  containers-storage            libio-pty-perl   libunibreak6                  podman
  crun                          libipc-run-perl  libxmlsec1-1                  postgresql-common-dev
  docker-compose                libiptcdata0     libxmlsec1-openssl1           python3-pluginbase
  fuse-overlayfs                libmbedcrypto16  linux-base-6.18.12+kali-amd64 python3-pysnmp4
  gccgo-14                      libpostal-data   linux-image-6.18.12+kali-amd64 python3-tz
  gccgo-14-x86-64-linux-gnu     libpostal1       netavark                      slirp4netns
  golang-github-containers-common libraw23t64    node-async-each               uidmap
  golang-github-containers-image libsdl2-classic node-is-binary-path
  libavc1394-0                  libsimdutf31     node-path-is-absolute
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 302
```

2. Preparing Secrets(Unencrypted - Insecure)

- Created a secrets file in plaintext:

```
┌──(kali㉿kali)-[~]
└─$ echo "API_KEY=super-secret-key-123
dquote> DB_PASSWORD=VeryStrongPassword" > secrets.env
```

3. Symmetric Encryption of Secrets (Password-Based)

- Encrypted the file using a password

```
┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ ls
README.md  secrets.env

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ gpg -c secrets.env

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ ls
README.md  secrets.env  secrets.env.gpg
```

4. Remove secrets.env

```
┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ rm secrets.env

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ ls
README.md  secrets.env.gpg
```

5. Decrypting the Secrets

```
┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ gpg secrets.env.gpg
gpg: WARNING: no command supplied.  Trying to guess what you mean ...
gpg: AES256.CFB encrypted data
gpg: encrypted with 1 passphrase

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ gpg -d secrets.env.gpg
gpg: AES256.CFB encrypted data
gpg: encrypted with 1 passphrase
API_KEY=super-secret-key-123
DB_PASSWORD=VeryStrongPassword
```

6. Asymmetric Encryption

```
┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ gpg --encrypt --recipient alice@example.com secrets.env
File 'secrets.env.gpg' exists. Overwrite? (y/N) y

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ ls
README.md  secrets.env  secrets.env.gpg
```

7. Using secrets in an Application (Simulation)

- Unset Api key and Db Password

```
┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ source secrets.env

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ echo $API_KEY
super-secret-key-123

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ unset API_KEY

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ unset DB_PASSWORD

┌──(kali㉿kali)-[~/cybersecurity/CS-Vaje/lab06]
└─$ ls
README.md  secrets.env  secrets.env.gpg
```

8. Multiple Users

Reflection

1. Why don't secrets belong in source code ?

SecretsSecrets in source code are exposed through sharing, repository leaks, or Git history. Automated scanners and bots discover them quickly, and a single leaked credential can compromise entire systems. Git history and forks often retain secrets even after removal. Store secrets separately, encrypted, and provide access through secure channels.

2. Symmetric vs Asymmetric encryption ?

Symmetric: one secret (a password or key) is used to both encrypt and decrypt. It’s simple and efficient but requires secure sharing and management of that single secret.

Asymmetric: uses a public key to encrypt and a private key to decrypt. You can share the public key openly, avoiding password distribution. Asymmetric is better suited for team workflows and reduces the risk of intercepted shared passwords..

3. What if we lose the private key ?

If you lose the private key, any data encrypted to that key is irrecoverable. GPG has no backdoor or recovery mechanism. Always keep secure, offline backups of private keys (and passphrases), or store them in a hardened key-vault solution.

4. How to handle this in a large enterprise ?

Use a purpose-built secrets manager (for example HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault). These systems provide centralized access control, automatic rotation, audit logging, short-lived dynamic credentials, and CI/CD integration. GPG is acceptable for personal use or small teams, but production environments typically require enterprise-grade secrets management for scale, compliance, and operational safety.